

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально-науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 3

з навчальної дисципліни

"Технології розробки корпоративних Web-додатків"

Виконав:

Студент 402 – ТН

Марков Іван Андрійович

Перевірив:

Тютюнник Петро Богданович

Полтава 2026

Лабораторна робота №3

Розробка серверної частини (Back-end) та API

Завдання

Мета роботи - реалізувати шар доступу до даних, бізнес-логіку та надати зовнішній інтерфейс для взаємодії з системою, використовуючи один із сучасних стандартів (REST, GraphQL або gRPC).

1. Реалізація шару доступу до даних (Data Access Layer)

- Налаштуйте підключення до бази даних через обрану ORM/ODM (Entity Framework, Hibernate, Mongoose, Drizzle тощо).
- Створіть моделі/сутності (Entities), що відповідають схемі БД з Лаб №2.
- Реалізуйте базові операції читання та запису (бажано через патерн Repository або ізольовані сервіси доступу до даних).

2. Реалізація бізнес-логіки (Service Layer)

- Створіть сервісний шар, який інкапсулює бізнес-правила.
- Вимога: API-контролери або резолвери не повинні містити складної логіки. Вони лише приймають дані, викликають сервіс і повертають результат клієнту.
- Застосуйте Dependency Injection (DI) для зв'язування компонентів.

3. Реалізація API (Оберіть один із варіантів) Виберіть підхід, який найкраще пасує до вашого тех стеку та вимог:

Варіант А: RESTful API

- Реалізуйте контролер для обраної сутності.
- Забезпечте підтримку HTTP методів: GET (List/Single), POST (Create), PUT/PATCH (Update), DELETE.
- Правильно використовуйте HTTP статус-коди (200, 201, 400, 404, 500).
- Документація: Swagger (OpenAPI).

4. Використання DTO (Data Transfer Objects)

- Незалежно від обраного протоколу, заборонено повертати клієнту "сирі" сутності бази даних.
- Використовуйте DTO (або message у gRPC, Type/Input у GraphQL) для передачі даних.
- Налаштуйте мапінг між сутностями БД та моделями API.

Хід роботи

Посилання на PR:

https://github.com/BanditFromSiege/TRVD_2026_402_TN_Markov_Labs/pull/3

Завдання №1.

Для виконання першого завдання, була створена папка SimpleLibrary.Infrastructure, в якій створено проєкт SimpleLibrary.Infrastructure, який і буде відповідати за Data Access Layer проєкту.

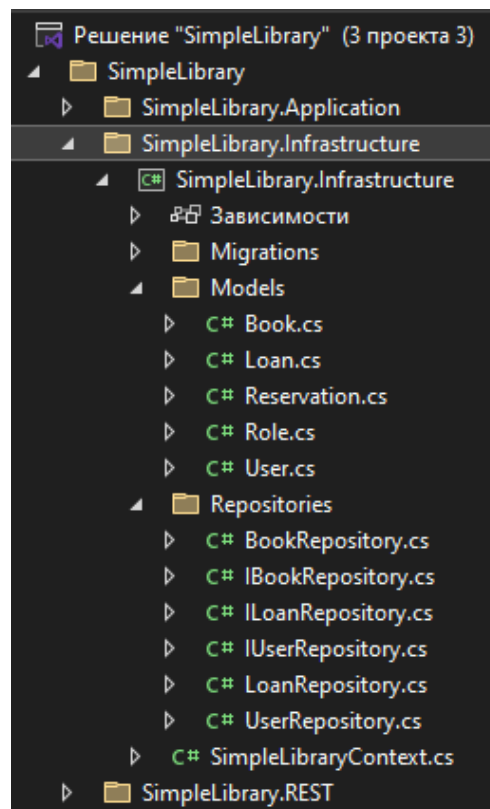


Рисунок 1 - Ієрархія файлів в проєкті SimpleLibrary.Infrastructure.

В папці Models було створено моделі відповідні до кожної таблиці в базі даних з лб №2. Ось їх програмний код.

Код файла Book.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SimpleLibrary.Infrastructure.Models
{
    public class Book
    {
        public int Id { get; set; }
        public string Title { get; set; } = null!;
        public string Author { get; set; } = null!;
        public string? Genre { get; set; } = null!;
        public string? Isbn { get; set; } = null!;

        public int TotalCopies { get; set; }
        public int AvailableCopies { get; set; }

        public ICollection<Loan> Loans { get; set; } = new List<Loan>();
        public ICollection<Reservation> Reservations { get; set; } = new
List<Reservation>();
    }
}
```

Код файла Loan.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SimpleLibrary.Infrastructure.Models
{
    public class Loan
    {
        public int Id { get; set; }

        public int UserId { get; set; }
        public User User { get; set; } = null!;

        public int BookId { get; set; }
        public Book Book { get; set; } = null!;

        public DateTime IssuedAt { get; set; }
        public DateTime DueDate { get; set; }
        public DateTime? ReturnedAt { get; set; }
    }
}
```

Код файла Reservation.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SimpleLibrary.Infrastructure.Models
{
```

```

public class Reservation
{
    public int Id { get; set; }

    public int UserId { get; set; }
    public User User { get; set; } = null!;

    public int BookId { get; set; }
    public Book Book { get; set; } = null!;

    public DateTime ReservedAt { get; set; }
    public string Status { get; set; } = null!;
}
}

```

Код файла Role.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SimpleLibrary.Infrastructure.Models
{
    public class Role
    {
        public int Id { get; set; }
        public string Name { get; set; } = null!;

        public ICollection<User> Users { get; set; } = new List<User>();
    }
}

```

Код файла User.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SimpleLibrary.Infrastructure.Models
{
    public class User
    {
        public int Id { get; set; }
        public string FullName { get; set; } = null!;
        public string Email { get; set; } = null!;
        public string PasswordHash { get; set; } = null!;

        public int RoleId { get; set; }
        public Role Role { get; set; } = null!;

        public DateTime CreatedAt { get; set; }

        public ICollection<Loan> Loans { get; set; } = new List<Loan>();
        public ICollection<Reservation> Reservations { get; set; } = new
List<Reservation>();
    }
}

```

В папці Repositories було створено репозиторії для моделей: User; Book; Loan, відповідно до вимог лб №2. Ось їх програмний код.

Програмний код UserRepository

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Infrastructure.Repositories
{
    public interface IUserRepository
    {
        Task<User?> GetByIdAsync(int id);
        Task<User?> GetByEmailAsync(string email);
        Task<Role?> GetRoleByIdAsync(int roleId);
        Task<IEnumerable<User>> GetAllAsync();
        Task AddAsync(User user);
        Task UpdateAsync(User user);
        Task DeleteAsync(User user);
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Infrastructure.Repositories
{
    public class UserRepository : IUserRepository
    {
        private readonly SimpleLibraryContext _context;

        public UserRepository(SimpleLibraryContext context)
        {
            _context = context;
        }

        public async Task<User?> GetByIdAsync(int id) =>
            await _context.Users.Include(u => u.Role).FirstOrDefaultAsync(u => u.Id ==
id);

        public async Task<User?> GetByEmailAsync(string email) =>
            await _context.Users.Include(u => u.Role).FirstOrDefaultAsync(u => u.Email
== email);

        public async Task<Role?> GetRoleByIdAsync(int roleId)
        {
            return await _context.Roles.FindAsync(roleId);
        }

        public async Task<IEnumerable<User>> GetAllAsync() =>
            await _context.Users.Include(u => u.Role).ToListAsync();

        public async Task AddAsync(User user)
        {
            await _context.Users.AddAsync(user);
            await _context.SaveChangesAsync();
        }
    }
}
```

```

    }

    public async Task UpdateAsync(User user)
    {
        _context.Users.Update(user);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(User user)
    {
        _context.Users.Remove(user);
        await _context.SaveChangesAsync();
    }
}

```

Программный код BookRepository

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Infrastructure.Repositories
{
    public interface IBookRepository
    {
        Task<Book?> GetByIdAsync(int id);
        Task<IEnumerable<Book>> GetAllAsync();
        Task<Book?> FindByTitleAndAuthorAsync(string title, string author);
        Task AddAsync(Book book);
        Task UpdateAsync(Book book);
        Task DeleteAsync(Book book);
    }
}

using Microsoft.EntityFrameworkCore;
using SimpleLibrary.Infrastructure.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SimpleLibrary.Infrastructure.Repositories
{
    public class BookRepository : IBookRepository
    {
        private readonly SimpleLibraryContext _context;

        public BookRepository(SimpleLibraryContext context)
        {
            _context = context;
        }

        public async Task<Book?> GetByIdAsync(int id) =>
            await _context.Books.FirstOrDefaultAsync(b => b.Id == id);

        public async Task<IEnumerable<Book>> GetAllAsync() =>
            await _context.Books.ToListAsync();

        public async Task<Book?> FindByTitleAndAuthorAsync(string title, string
author)
        {
            return await _context.Books

```

```

        .FirstOrDefaultAsync(b => b.Title == title && b.Author == author);
    }

    public async Task AddAsync(Book book)
    {
        await _context.Books.AddAsync(book);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(Book book)
    {
        _context.Books.Update(book);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(Book book)
    {
        _context.Books.Remove(book);
        await _context.SaveChangesAsync();
    }
}
}

```

Программный код LoanRepository

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Infrastructure.Repositories
{
    public interface ILoanRepository
    {
        Task<Loan?> GetByIdAsync(int id);
        Task<IEnumerable<Loan>> GetAllAsync();
        Task AddAsync(Loan loan);
        Task UpdateAsync(Loan loan);
        Task DeleteAsync(Loan loan);
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Infrastructure.Repositories
{
    public class LoanRepository : ILoanRepository
    {
        private readonly SimpleLibraryContext _context;

        public LoanRepository(SimpleLibraryContext context)
        {
            _context = context;
        }

        public async Task<Loan?> GetByIdAsync(int id) =>
            await _context.Loans
                .Include(l => l.User)
                .Include(l => l.Book)

```



```

        .FirstOrDefaultAsync(l => l.Id == id);

public async Task<IEnumerable<Loan>> GetAllAsync() =>
    await _context.Loans
        .Include(l => l.User)
        .Include(l => l.Book)
        .ToListAsync();

public async Task AddAsync(Loan loan)
{
    await _context.Loans.AddAsync(loan);
    await _context.SaveChangesAsync();
}

public async Task UpdateAsync(Loan loan)
{
    _context.Loans.Update(loan);
    await _context.SaveChangesAsync();
}

public async Task DeleteAsync(Loan loan)
{
    _context.Loans.Remove(loan);
    await _context.SaveChangesAsync();
}
}
}

```

В файлі SimpleLibraryContext.cs було створено код, який завдяки EF Core створює міграції та базу даних SQLite в папці з проектом SimpleLibrary.REST. Ось програмний код файлу.

Програмний код SimpleLibraryContext

```

using System.Collections.Generic;
using System.Reflection.Emit;
using Microsoft.EntityFrameworkCore;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Infrastructure
{
    public class SimpleLibraryContext : DbContext
    {
        public SimpleLibraryContext() { }
        public SimpleLibraryContext(DbContextOptions<SimpleLibraryContext> options)
            : base(options) { }

        public DbSet<User> Users => Set<User>();
        public DbSet<Role> Roles => Set<Role>();
        public DbSet<Book> Books => Set<Book>();
        public DbSet<Loan> Loans => Set<Loan>();
        public DbSet<Reservation> Reservations => Set<Reservation>();

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<User>()
                .HasIndex(u => u.Email)
                .IsUnique();

            modelBuilder.Entity<Book>()
                .HasIndex(b => b.Isbn)
                .IsUnique();

            modelBuilder.Entity<Role>()

```

```

        .HasIndex(r => r.Name)
        .IsUnique();
    }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (!optionsBuilder.IsConfigured)
        {
            optionsBuilder.UseSqlite("Data Source=SimpleLibrary.db");
        }
    }
}

```

Ось ER – діаграма створеної бази даних.

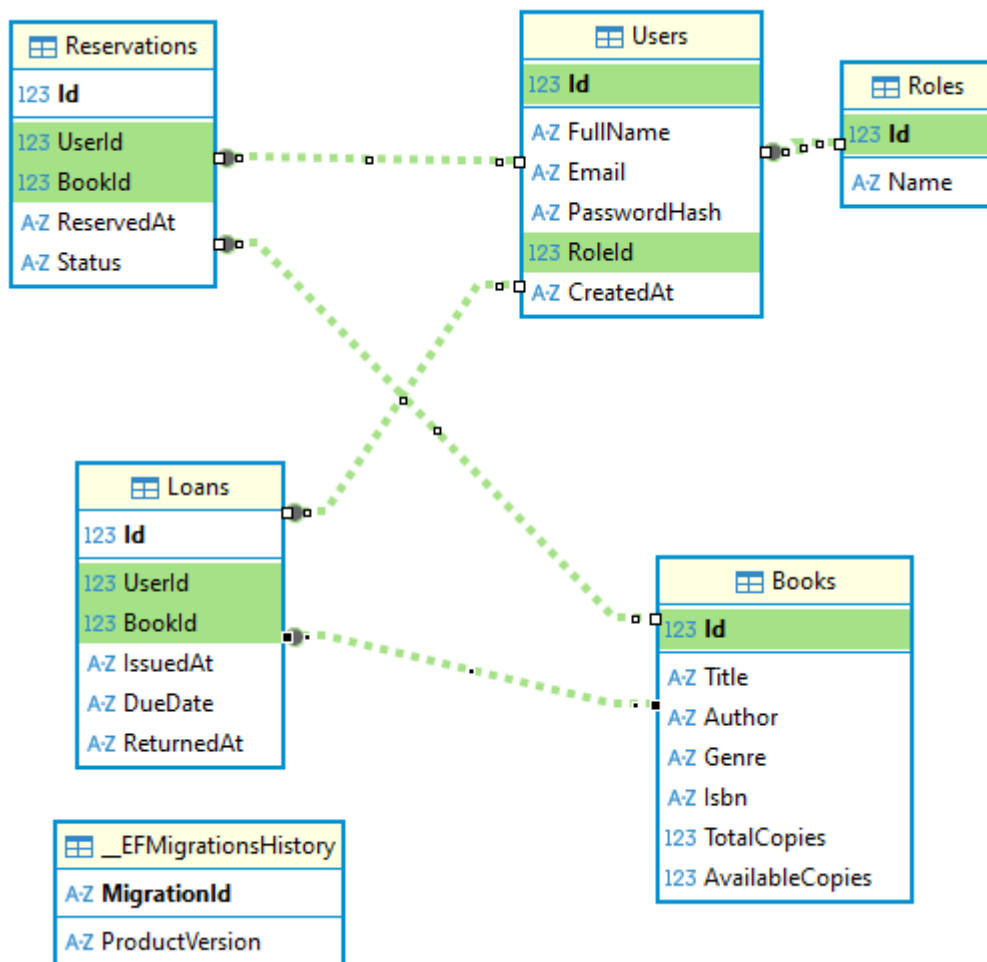


Рисунок 2 - ER – діаграма бази даних SimpleLibrary.

Також виведемо всі параметри кожної таблиці.

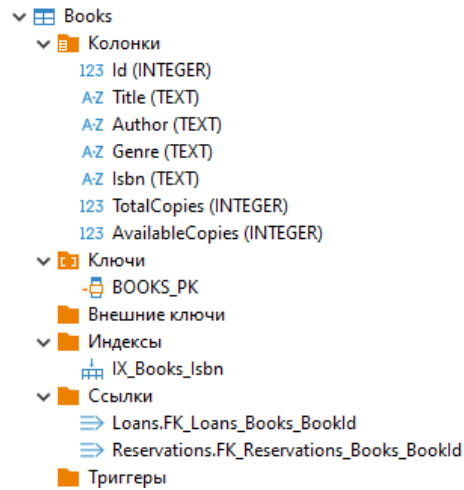


Рисунок 3 – Параметры таблиці Books.

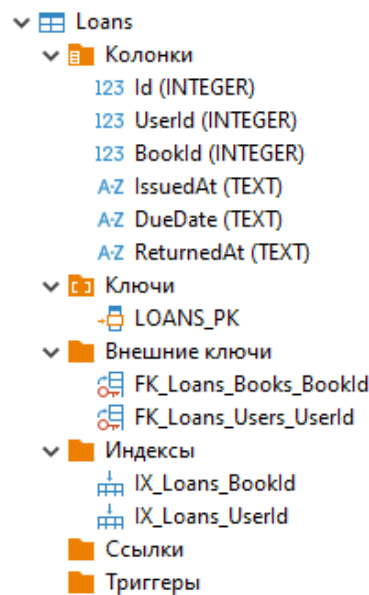


Рисунок 4 – Параметры таблиці Loans.

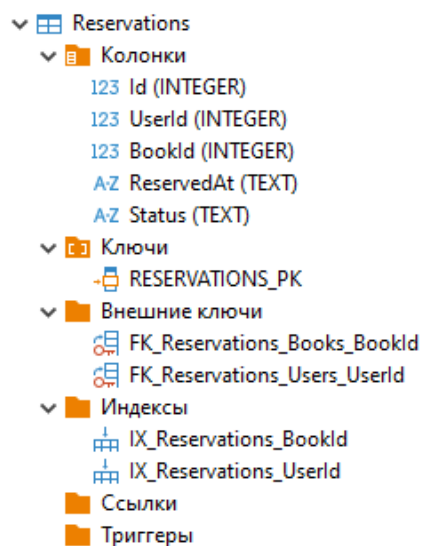


Рисунок 5 – Параметры таблиці Reservations.

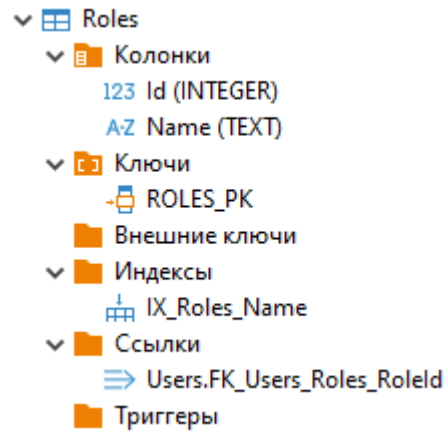


Рисунок 6 – Параметры таблиці Roles.

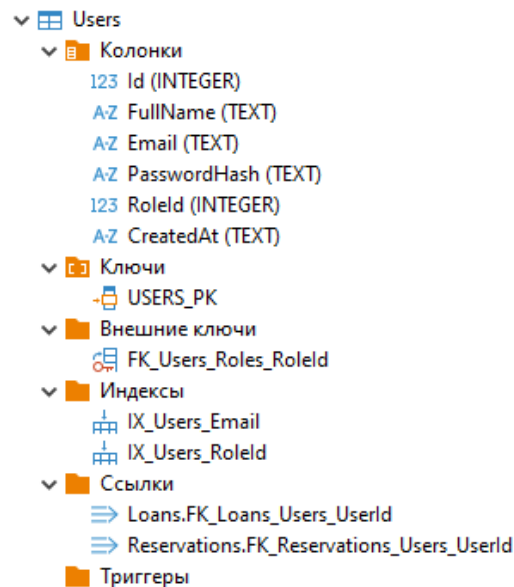


Рисунок 7 – Параметры таблиці Users.

Завдання №2.

Для виконання другого завдання, була створена папка SimpleLibrary.Application, в якій створено проєкт SimpleLibrary.Application, який і буде відповідати за бізнес - логіку проєкту.

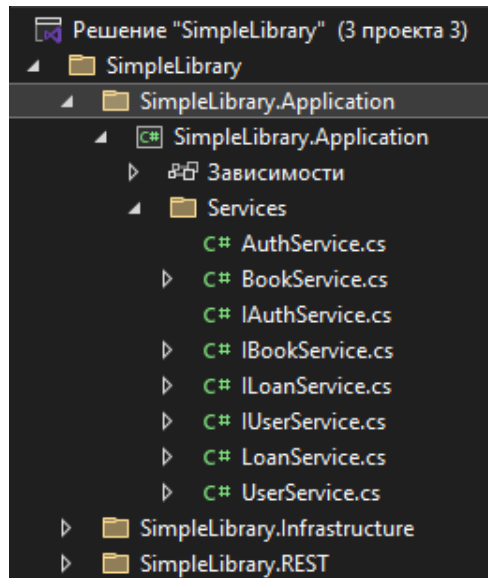


Рисунок 8 – Ієрархія файлів в проєкті SimpleLibrary.Application.

В папці Services було створено сервіси відповідно до вимог лб №2. Ось їх програмний код.

Програмний код UserService

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Application.Services
{
    public interface IUserService
    {
        Task<User?> GetUserByIdAsync(int id);
        Task<User?> GetUserByEmailAsync(string email);
        Task<IEnumerable<User>> GetAllUsersAsync();
        Task<User> CreateUserAsync(User user, string password);
        Task<bool> UpdateUserAsync(int id, User user);
        Task<bool> DeleteUserAsync(int id);
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;
using SimpleLibrary.Infrastructure.Repositories;

namespace SimpleLibrary.Application.Services
{
    public class UserService : IUserService
    {
        private readonly IUserRepository _userRepository;

        public UserService(IUserRepository userRepository)
        {

```

```

        _userRepository = userRepository;
    }

    public async Task<User?> GetUserByEmailAsync(string email)
    {
        if (string.IsNullOrEmpty(email))
            return null;

        var user = await _userRepository.GetByEmailAsync(email);
        if (user == null)
            return null;

        return user;
    }

    public async Task<User?> GetUserByIdAsync(int id)
    {
        var user = await _userRepository.GetByIdAsync(id);
        if (user == null)
            return null;

        return user;
    }

    public async Task<IEnumerable<User>> GetAllUsersAsync()
    {
        var users = await _userRepository.GetAllAsync();
        return users;
    }

    public async Task<User> CreateUserAsync(User user, string password)
    {
        //For lab4
        //user.PasswordHash = BCrypt.Net.BCrypt.HashPassword(password);
        user.PasswordHash = password;
        user.CreatedAt = DateTime.UtcNow;

        await _userRepository.AddAsync(user);

        user.Role = await _userRepository.GetRoleByIdAsync(user.RoleId)
            ?? throw new Exception("Role with this Id not found");

        return user;
    }

    public async Task<bool> UpdateUserAsync(int id, User updatedUser)
    {
        var existingUser = await _userRepository.GetByIdAsync(id);
        if (existingUser == null)
            return false;

        existingUser.FullName = updatedUser.FullName;
        existingUser.Email = updatedUser.Email;

        await _userRepository.UpdateAsync(existingUser);
        return true;
    }

    public async Task<bool> DeleteUserAsync(int id)
    {
        var user = await _userRepository.GetByIdAsync(id);
        if (user == null)
            return false;

        await _userRepository.DeleteAsync(user);
        return true;
    }
}

```

```
}
```

Програмный код BookService

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Application.Services
{
    public interface IBookService
    {
        Task<IEnumerable<Book>> GetAllBooksAsync();
        Task<Book?> GetBookByIdAsync(int id);
        Task<Book> CreateBookAsync(Book book);
        Task<Book?> UpdateBookAsync(int id, Book book);
        Task<bool> DeleteBookAsync(int id);
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;
using SimpleLibrary.Infrastructure.Repositories;

namespace SimpleLibrary.Application.Services
{
    public class BookService : IBookService
    {
        private readonly IBookRepository _bookRepository;

        public BookService(IBookRepository bookRepository)
        {
            _bookRepository = bookRepository;
        }

        public async Task<IEnumerable<Book>> GetAllBooksAsync()
        {
            return await _bookRepository.GetAllAsync();
        }

        public async Task<Book?> GetBookByIdAsync(int id)
        {
            return await _bookRepository.GetByIdAsync(id);
        }

        public async Task<Book> CreateBookAsync(Book book)
        {
            var existing = await _bookRepository.FindByTitleAndAuthorAsync(book.Title,
book.Author);
            if (existing != null)
                throw new Exception("Book with this title and author already exists");

            await _bookRepository.AddAsync(book);
            return book;
        }

        public async Task<Book?> UpdateBookAsync(int id, Book book)
        {
            var existing = await _bookRepository.GetByIdAsync(id);
            if (existing == null)
```

```

        return null;

        var duplicate = await
_bookRepository.FindByTitleAndAuthorAsync(book.Title, book.Author);
        if (duplicate != null && duplicate.Id != id)
            throw new Exception("Book with this title and author already exists");

        int borrowedCopies = existing.TotalCopies - existing.AvailableCopies;

        if (book.TotalCopies < borrowedCopies)
            throw new Exception("TotalCopies cannot be less than borrowed
copies");

        int difference = book.TotalCopies - existing.TotalCopies;

        existing.Title = book.Title;
        existing.Author = book.Author;
        existing.Genre = book.Genre;
        existing.Isbn = book.Isbn;
        existing.TotalCopies = book.TotalCopies;
        existing.AvailableCopies += difference;

        await _bookRepository.UpdateAsync(existing);
        return existing;
    }

    public async Task<bool> DeleteBookAsync(int id)
    {
        var book = await _bookRepository.GetByIdAsync(id);
        if (book == null) return false;

        await _bookRepository.DeleteAsync(book);
        return true;
    }
}

```

Програмный код LoanService

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;

namespace SimpleLibrary.Application.Services
{
    public interface ILoanService
    {
        Task<IEnumerable<Loan>> GetAllLoansAsync();
        Task<Loan?> GetLoanByIdAsync(int id);
        Task<Loan> CreateLoanAsync(Loan loan);
        Task<Loan?> UpdateLoanAsync(int id, Loan loan);
        Task<bool> DeleteLoanAsync(int id);
        Task<bool> CheckBookAvailabilityAsync(int bookId);
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SimpleLibrary.Infrastructure.Models;
using SimpleLibrary.Infrastructure.Repositories;

```



```

namespace SimpleLibrary.Application.Services
{
    public class LoanService : ILoanService
    {
        private readonly ILoanRepository _loanRepository;
        private readonly IBookRepository _bookRepository;

        public LoanService(ILoanRepository loanRepository, IBookRepository
bookRepository)
        {
            _loanRepository = loanRepository;
            _bookRepository = bookRepository;
        }

        public async Task<IEnumerable<Loan>> GetAllLoansAsync()
        {
            var loans = await _loanRepository.GetAllAsync();
            return loans;
        }

        public async Task<Loan?> GetLoanByIdAsync(int id)
        {
            var loan = await _loanRepository.GetByIdAsync(id);
            if (loan == null)
                return null;

            return loan;
        }

        public async Task<Loan> CreateLoanAsync(Loan loan)
        {
            var book = await _bookRepository.GetByIdAsync(loan.BookId);
            if (book == null || book.AvailableCopies <= 0)
                throw new Exception("Book is not available");

            book.AvailableCopies--;

            loan.IssuedAt = DateTime.UtcNow;

            await _loanRepository.AddAsync(loan);
            await _bookRepository.UpdateAsync(book);

            return loan;
        }

        public async Task<Loan?> UpdateLoanAsync(int id, Loan loan)
        {
            var existing = await _loanRepository.GetByIdAsync(id);
            if (existing == null)
                return null;

            bool isReturningNow = existing.ReturnedAt == null && loan.ReturnedAt !=
null;

            if (loan.DueDate != default)
            {
                existing.DueDate = loan.DueDate;
            }

            if (loan.ReturnedAt != null)
            {
                existing.ReturnedAt = loan.ReturnedAt;
            }

            if (isReturningNow)
            {
                var book = await _bookRepository.GetByIdAsync(existing.BookId);
                if (book != null)

```

```

        {
            book.AvailableCopies++;
            await _bookRepository.UpdateAsync(book);
        }
    }

    await _loanRepository.UpdateAsync(existing);

    return existing;
}

public async Task<bool> DeleteLoanAsync(int id)
{
    var loan = await _loanRepository.GetByIdAsync(id);
    if (loan == null)
        return false;

    await _loanRepository.DeleteAsync(loan);
    return true;
}

public async Task<bool> CheckBookAvailabilityAsync(int bookId)
{
    var book = await _bookRepository.GetByIdAsync(bookId);
    return book != null && book.AvailableCopies > 0;
}
}
}

```

Завдання №3.

Для виконання третього завдання, була створена папка SimpleLibrary.REST, в якій створено проєкт SimpleLibrary.REST, який і буде відповідати за API проєкту.

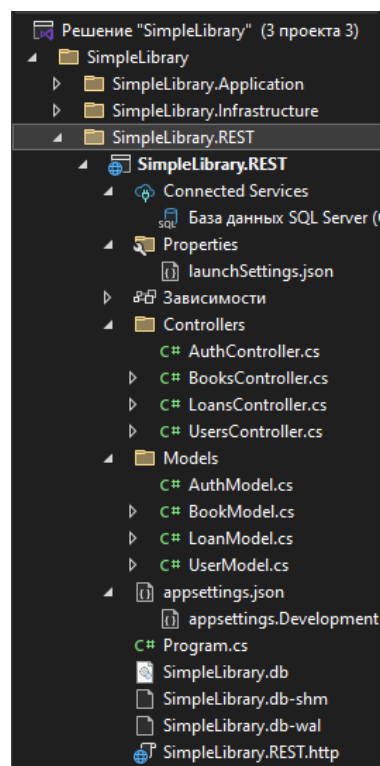


Рисунок 9 – Ієрархія файлів в проєкті SimpleLibrary.REST.

В папці Controllers було створено контролери відповідно до вимог лб №2.
Ось їх програмний код.

Код файлу UsersController.cs

```
using Microsoft.AspNetCore.Mvc;
using SimpleLibrary.Application.Services;
using SimpleLibrary.Infrastructure.Models;
using SimpleLibrary.REST.Models;

namespace SimpleLibrary.REST.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class UsersController : ControllerBase
    {
        private readonly IUserService _userService;

        public UsersController(IUserService userService)
        {
            _userService = userService;
        }

        // GET: api/users
        [HttpGet]
        public async Task<ActionResult<IEnumerable<UserResponseModel>>> GetAllUsers()
        {
            var users = await _userService.GetAllUsersAsync();

            var result = users.Select(MapToResponse);

            return Ok(result);
        }

        // GET: api/users/{id}
        [HttpGet("{id}")]
        public async Task<ActionResult<UserResponseModel>> GetUserById(int id)
        {
            var user = await _userService.GetUserByIdAsync(id);
            if (user == null)
                return NotFound();

            return Ok(MapToResponse(user));
        }

        // POST: api/users
        [HttpPost]
        public async Task<ActionResult<UserResponseModel>> CreateUser([FromBody]
UserCreateModel model)
        {
            var validRoleIds = new[] { 1, 2, 3 }; // 1 = User, 2 = Librarian, 3 =
Admin
            if (!validRoleIds.Contains(model.RoleId))
            {
                return BadRequest("Invalid role");
            }

            var user = new User
            {
                FullName = model.FullName,
                Email = model.Email,
                RoleId = model.RoleId,
                CreatedAt = DateTime.UtcNow
            };
        }
    }
}
```

```

        var created = await _userService.CreateUserAsync(user, model.Password);

        var response = new UserResponseModel
        {
            Id = created.Id,
            FullName = created.FullName,
            Email = created.Email,
            Role = created.Role.Name,
            CreatedAt = created.CreatedAt
        };

        return CreatedAtAction(
            nameof(GetUserById),
            new { id = created.Id },
            response
        );
    }

    // PUT: api/users/{id}
    [HttpPut("{id}")]
    public async Task<IActionResult> UpdateUser(int id, [FromBody] UserUpdateModel
model)
    {
        var user = new User
        {
            FullName = model.FullName,
            Email = model.Email
        };

        var updated = await _userService.UpdateUserAsync(id, user);

        if (!updated)
            return NotFound();

        return NoContent();
    }

    // DELETE: api/users/{id}
    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteUser(int id)
    {
        var deleted = await _userService.DeleteUserAsync(id);

        if (!deleted)
            return NotFound();

        return NoContent();
    }

    private static UserResponseModel MapToResponse(User user)
    {
        return new UserResponseModel
        {
            Id = user.Id,
            FullName = user.FullName,
            Email = user.Email,
            Role = user.Role?.Name ?? "",
            CreatedAt = user.CreatedAt
        };
    }
}

```

Код файла BooksController.cs

```
using Microsoft.AspNetCore.Mvc;
using SimpleLibrary.Application.Services;
using SimpleLibrary.Infrastructure.Models;
using SimpleLibrary.REST.Models;

namespace SimpleLibrary.REST.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class BooksController : ControllerBase
    {
        private readonly IBookService _bookService;

        public BooksController(IBookService bookService)
        {
            _bookService = bookService;
        }

        // GET: api/books
        [HttpGet]
        public async Task<ActionResult<IEnumerable<BookResponseModel>>> GetAll()
        {
            var books = await _bookService.GetAllBooksAsync();

            var result = books.Select(MapToResponse);

            return Ok(result);
        }

        // GET: api/books/{id}
        [HttpGet("{id}")]
        public async Task<ActionResult<BookResponseModel>> GetById(int id)
        {
            var book = await _bookService.GetBookByIdAsync(id);

            if (book == null)
                return NotFound();

            return Ok(MapToResponse(book));
        }

        // POST: api/books
        [HttpPost]
        public async Task<ActionResult<BookResponseModel>> Create([FromBody]
BookCreateModel model)
        {
            var book = new Book
            {
                Title = model.Title,
                Author = model.Author,
                Isbn = model.ISBN,
                Genre = model.Genre,
                TotalCopies = model.TotalCopies,
                AvailableCopies = model.TotalCopies
            };

            var created = await _bookService.CreateBookAsync(book);

            return CreatedAtAction(
                nameof(GetById),
                new { id = created.Id },
                MapToResponse(created)
            );
        }
    }
}
```

```

// PUT: api/books/{id}
[HttpPut("{id}")]
public async Task<ActionResult<BookResponseModel>> Update(int id, [FromBody]
BookUpdateModel model)
{
    var book = new Book
    {
        Title = model.Title,
        Author = model.Author,
        Isbn = model.ISBN,
        Genre = model.Genre,
        TotalCopies = model.TotalCopies
    };

    var updated = await _bookService.UpdateBookAsync(id, book);

    if (updated == null)
        return NotFound();

    return Ok(MapToResponse(updated));
}

// DELETE: api/books/{id}
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var deleted = await _bookService.DeleteBookAsync(id);
    if (!deleted)
        return NotFound();

    return NoContent();
}

private static BookResponseModel MapToResponse(Book book)
{
    return new BookResponseModel
    {
        Id = book.Id,
        Title = book.Title,
        Author = book.Author,
        ISBN = book.Isbn ?? string.Empty,
        Genre = book.Genre ?? string.Empty,
        AvailableCopies = book.AvailableCopies
    };
}
}
}

```

Код файлу LoansController.cs

```

using Microsoft.AspNetCore.Mvc;
using SimpleLibrary.Application.Services;
using SimpleLibrary.Infrastructure.Models;
using SimpleLibrary.REST.Models;

namespace SimpleLibrary.REST.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class LoansController : ControllerBase
    {
        private readonly ILoanService _loanService;

        public LoansController(ILoanService loanService)
        {
            _loanService = loanService;
        }
    }
}

```

```

// GET: api/loans
[HttpGet]
public async Task<ActionResult<IEnumerable<LoanResponseModel>>> GetAll()
{
    var loans = await _loanService.GetAllLoansAsync();
    var result = loans.Select(MapToResponse);
    return Ok(result);
}

// GET: api/loans/{id}
[HttpGet("{id}")]
public async Task<ActionResult<LoanResponseModel>> GetById(int id)
{
    var loan = await _loanService.GetLoanByIdAsync(id);
    if (loan == null)
        return NotFound();

    return Ok(MapToResponse(loan));
}

// POST: api/loans
[HttpPost]
public async Task<ActionResult<LoanResponseModel>> Create([FromBody]
LoanCreateModel model)
{
    try
    {
        var loan = new Loan
        {
            UserId = model.UserId,
            BookId = model.BookId,
            DueDate = model.DueDate,
            IssuedAt = DateTime.UtcNow
        };

        var created = await _loanService.CreateLoanAsync(loan);
        return CreatedAtAction(nameof(GetById), new { id = created.Id },
MapToResponse(created));
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}

// PUT: api/loans/{id}
[HttpPut("{id}")]
public async Task<ActionResult<LoanResponseModel>> Update(int id, [FromBody]
LoanUpdateModel model)
{
    var loan = new Loan
    {
        ReturnedAt = model.ReturnDate
    };

    var updated = await _loanService.UpdateLoanAsync(id, loan);
    if (updated == null)
        return NotFound();

    return Ok(MapToResponse(updated));
}

// DELETE: api/loans/{id}
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var deleted = await _loanService.DeleteLoanAsync(id);
}

```

```

        if (!deleted)
            return NotFound();

        return NoContent();
    }

    private static LoanResponseModel MapToResponse(Loan loan)
    {
        return new LoanResponseModel
        {
            Id = loan.Id,
            UserId = loan.UserId,
            BookId = loan.BookId,
            LoanDate = loan.IssuedAt,
            DueDate = loan.DueDate,
            ReturnDate = loan.ReturnedAt
        };
    }
}
}

```

В файлі Program.cs було створено код, який підключає репозиторії, сервіси, контролери та Swagger. Ось програмний код файлу.

Код файлу Program.cs

```

using Microsoft.EntityFrameworkCore;
using SimpleLibrary.Infrastructure;
using SimpleLibrary.Infrastructure.Repositories;
using SimpleLibrary.Application.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<SimpleLibraryContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")
    ?? "Data Source=SimpleLibrary.db"));

builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IBookRepository, BookRepository>();
builder.Services.AddScoped<ILoanRepository, LoanRepository>();

builder.Services.AddScoped<IUserService, UserService>();
builder.Services.AddScoped<IBookService, BookService>();
builder.Services.AddScoped<ILoanService, LoanService>();
//builder.Services.AddScoped<IAuthService, AuthService>();

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();

```


Завдання №4.

Для виконання четвертого завдання, в папці SimpleLibrary.REST була створена папка Models, в якій створено DTO моделі для контролерів проєкту SimpleLibrary.REST. Ось програмний код цих моделей.

Код файлу UserModel.cs

```
namespace SimpleLibrary.REST.Models
{
    public class UserCreateModel
    {
        public string FullName { get; set; } = null!;
        public string Email { get; set; } = null!;
        public string Password { get; set; } = null!;
        public int RoleId { get; set; }
    }
    public class UserUpdateModel
    {
        public string FullName { get; set; } = null!;
        public string Email { get; set; } = null!;
    }

    public class UserResponseModel
    {
        public int Id { get; set; }

        public string FullName { get; set; } = null!;

        public string Email { get; set; } = null!;

        public string Role { get; set; } = null!;

        public DateTime CreatedAt { get; set; }
    }
}
```

Код файлу BookModel.cs

```
namespace SimpleLibrary.REST.Models
{
    public class BookCreateModel
    {
        public string Title { get; set; } = null!;

        public string Author { get; set; } = null!;

        public string ISBN { get; set; } = null!;

        public string Genre { get; set; } = null!;

        public int TotalCopies { get; set; }
    }

    public class BookUpdateModel
    {
        public string Title { get; set; } = null!;

        public string Author { get; set; } = null!;

        public string ISBN { get; set; } = null!;
    }
}
```

```

        public string Genre { get; set; } = null!;

        public int TotalCopies { get; set; }
    }

    public class BookResponseModel
    {
        public int Id { get; set; }

        public string Title { get; set; } = null!;

        public string Author { get; set; } = null!;

        public string ISBN { get; set; } = null!;

        public string Genre { get; set; } = null!;

        public int AvailableCopies { get; set; }
    }
}

```

Код файлу LoanModel.cs

```

namespace SimpleLibrary.REST.Models
{
    public class LoanCreateModel
    {
        public int UserId { get; set; }

        public int BookId { get; set; }

        public DateTime DueDate { get; set; }
    }

    public class LoanUpdateModel
    {
        public DateTime? ReturnDate { get; set; }
    }

    public class LoanResponseModel
    {
        public int Id { get; set; }

        public int UserId { get; set; }

        public int BookId { get; set; }

        public DateTime LoanDate { get; set; }

        public DateTime DueDate { get; set; }

        public DateTime? ReturnDate { get; set; }
    }
}

```

Мапінг реалізований завдяки методам MapToResponse в коді кожного контролера.

Приклади застосування програми через Swagger.

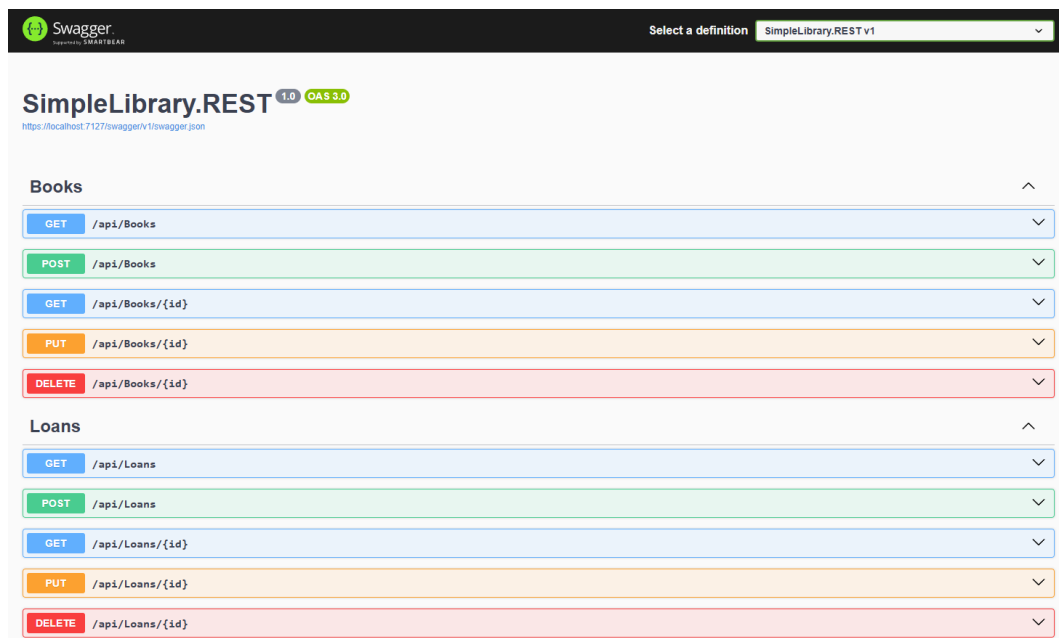


Рисунок 10 – Вікно Swagger.

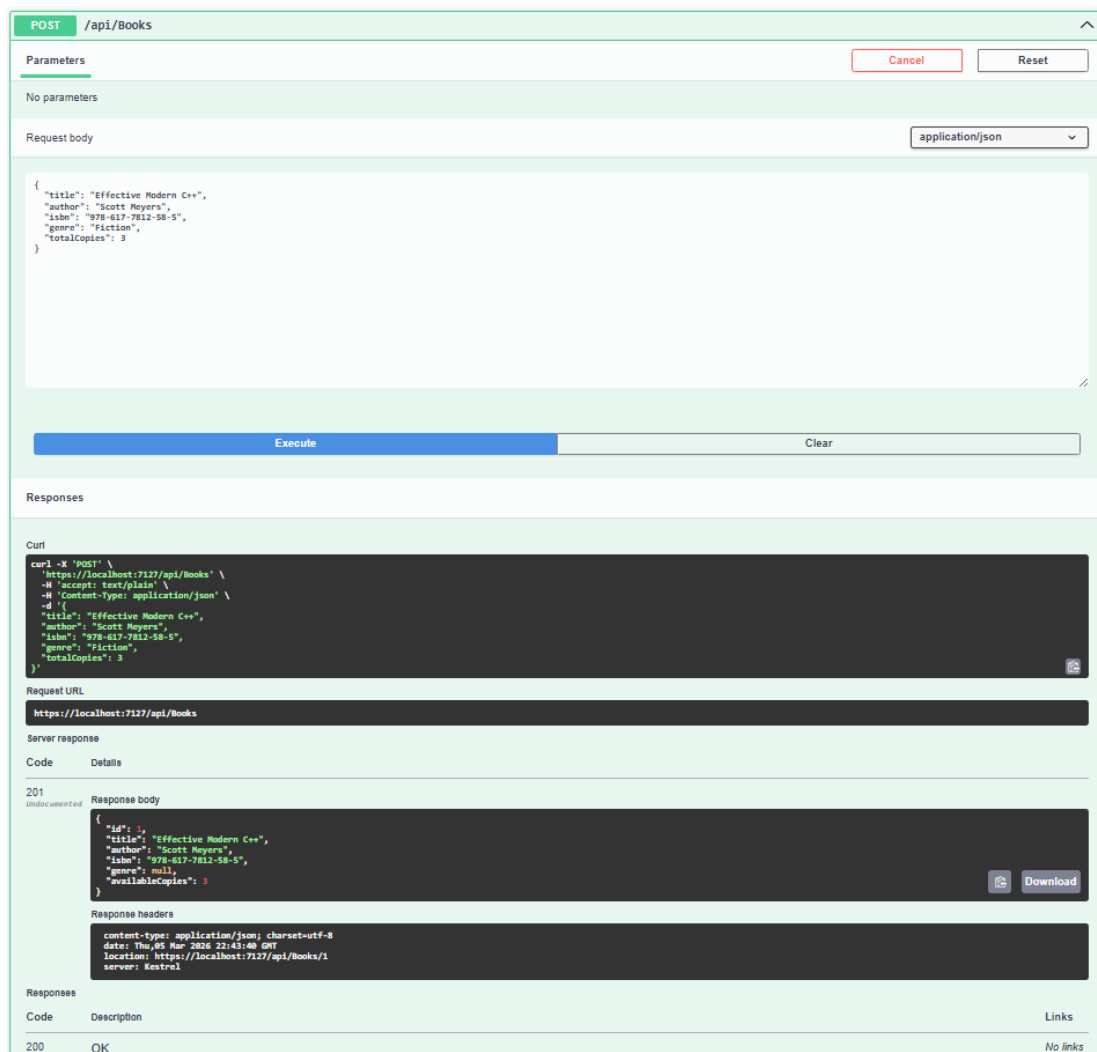


Рисунок 11 – Приклад застосування команди POST для Books.

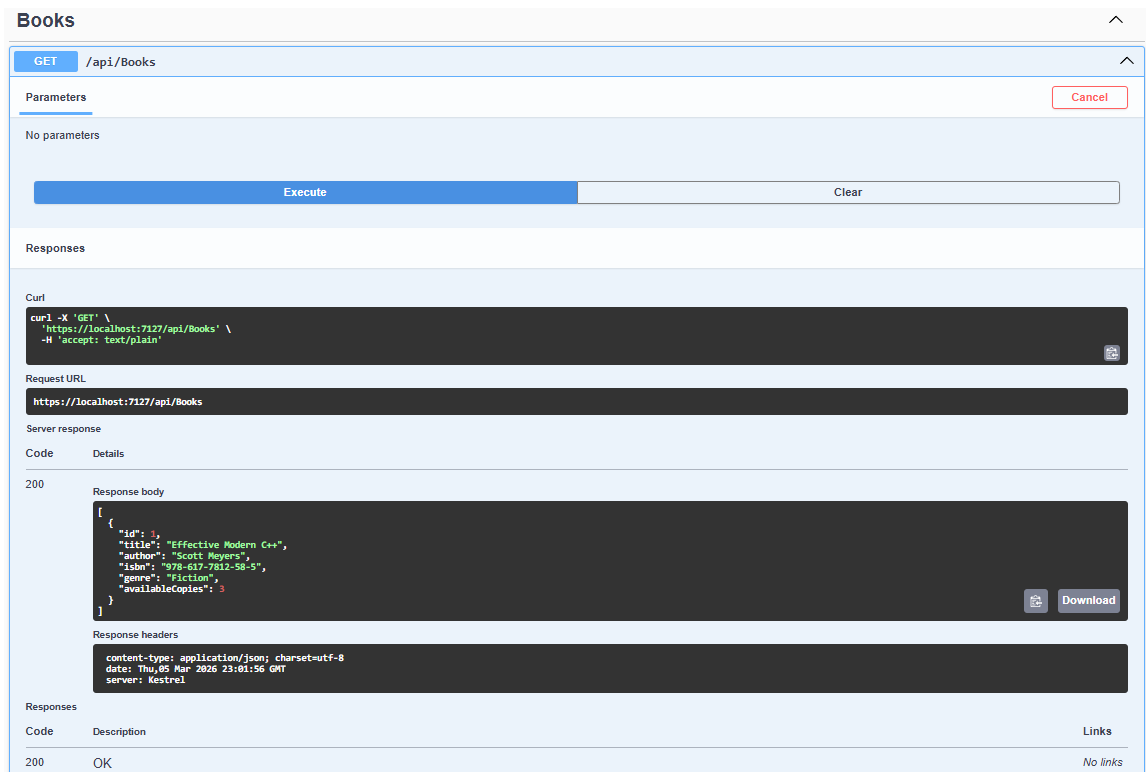


Рисунок 12 – Приклад застосування команди GET для Books.

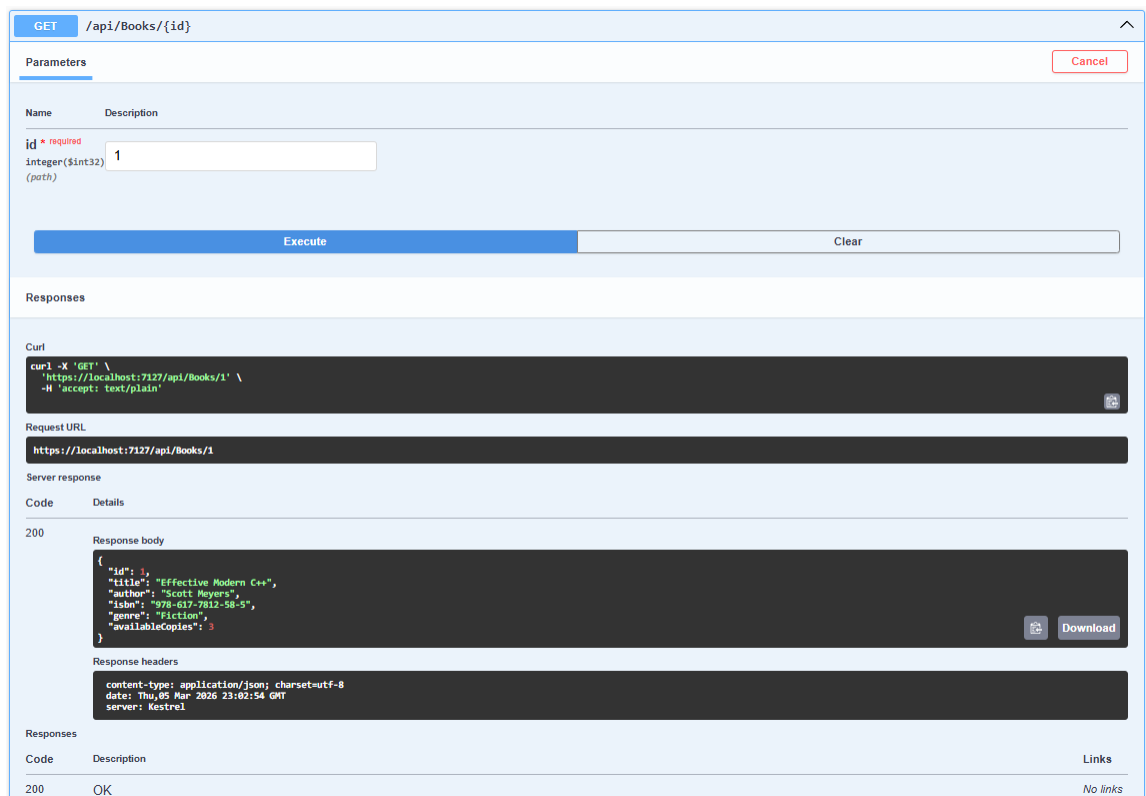


Рисунок 13 – Приклад застосування команди GET {id} для Books.

PUT

/api/Books/{id}

Cancel

Reset

Parameters

Name	Description
id * required	
integer(int32)	1
(path)	

Request body

application/json

```
{
  "title": "Effective Modern C++",
  "author": "Scott Meyers",
  "isbn": "978-617-7812-58-5",
  "genre": "Computer Science",
  "totalCopies": 3
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  https://localhost:7127/api/Books/1 \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Effective Modern C++",
    "author": "Scott Meyers",
    "isbn": "978-617-7812-58-5",
    "genre": "Computer Science",
    "totalCopies": 3
  }'
```

Request URL

https://localhost:7127/api/Books/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "title": "Effective Modern C++", "author": "Scott Meyers", "isbn": "978-617-7812-58-5", "genre": "Computer Science", "availableCopies": 3 }</pre> Download

Рисунок 14 – Приклад застосування команди PUT {id} для Books.

POST

/api/Books

Cancel

Reset

Parameters

No parameters

Request body

application/json

```
{
  "title": "The Algorithm Design Manual",
  "author": "Steven S. Skiena",
  "isbn": "978-5-9775-0568-4",
  "genre": "Computer Science",
  "totalCopies": 1
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  https://localhost:7127/api/Books \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "The Algorithm Design Manual",
    "author": "Steven S. Skiena",
    "isbn": "978-5-9775-0568-4",
    "genre": "Computer Science",
    "totalCopies": 1
  }'
```

Request URL

https://localhost:7127/api/Books

Server response

Code	Details
201	Response body <pre>{ "id": 1, "title": "The Algorithm Design Manual", "author": "Steven S. Skiena", "isbn": "978-5-9775-0568-4", "genre": "Computer Science", "availableCopies": 1 }</pre> Download

Рисунок 15 – Створення тимчасової книги для видалення.

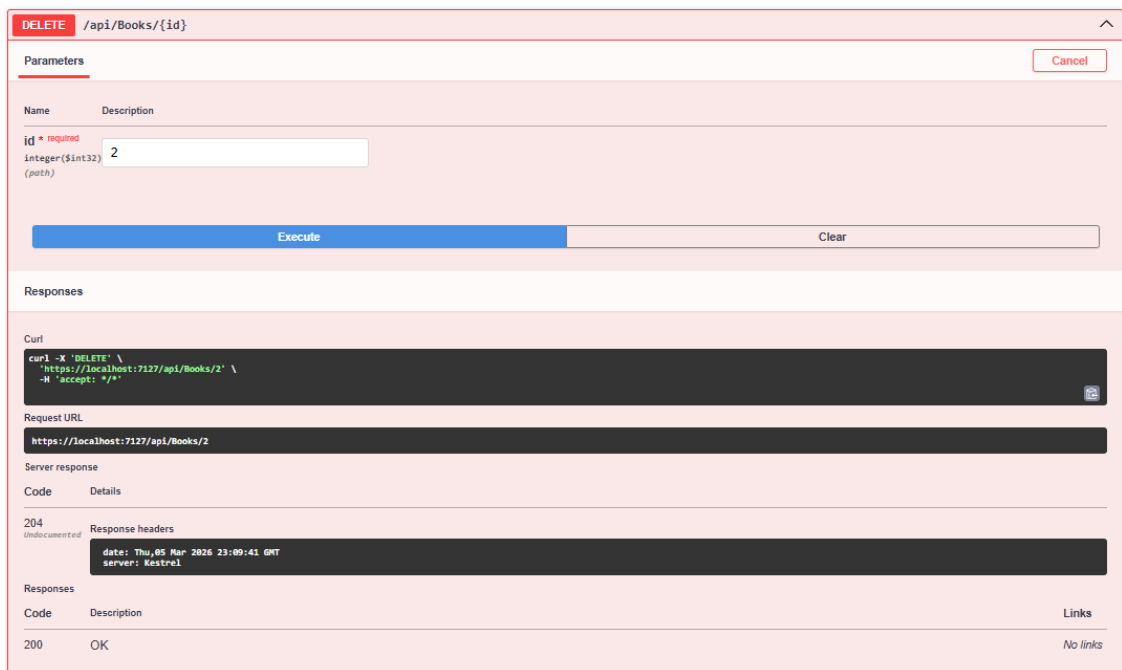


Рисунок 16 – Приклад застосування команди DELETE {id} для Books.

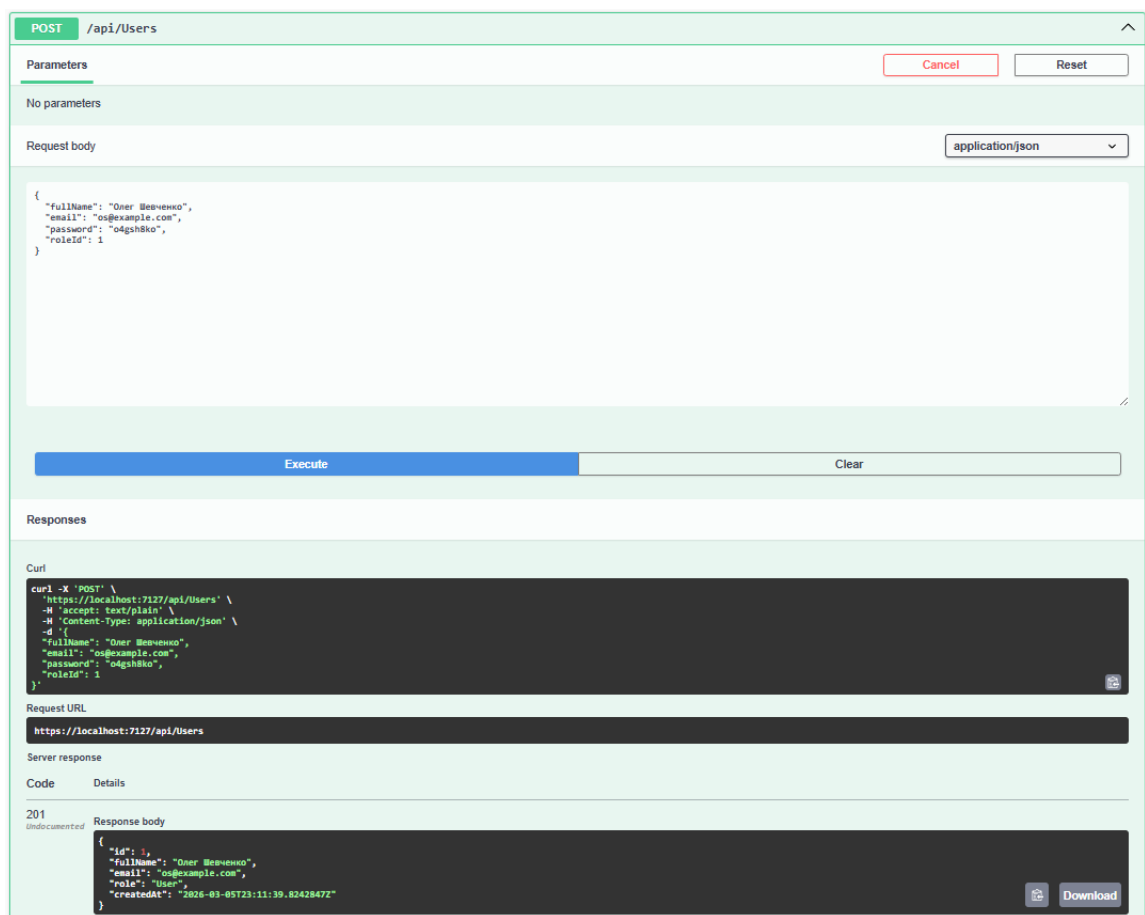


Рисунок 17 – Приклад застосування команди POST для Users.

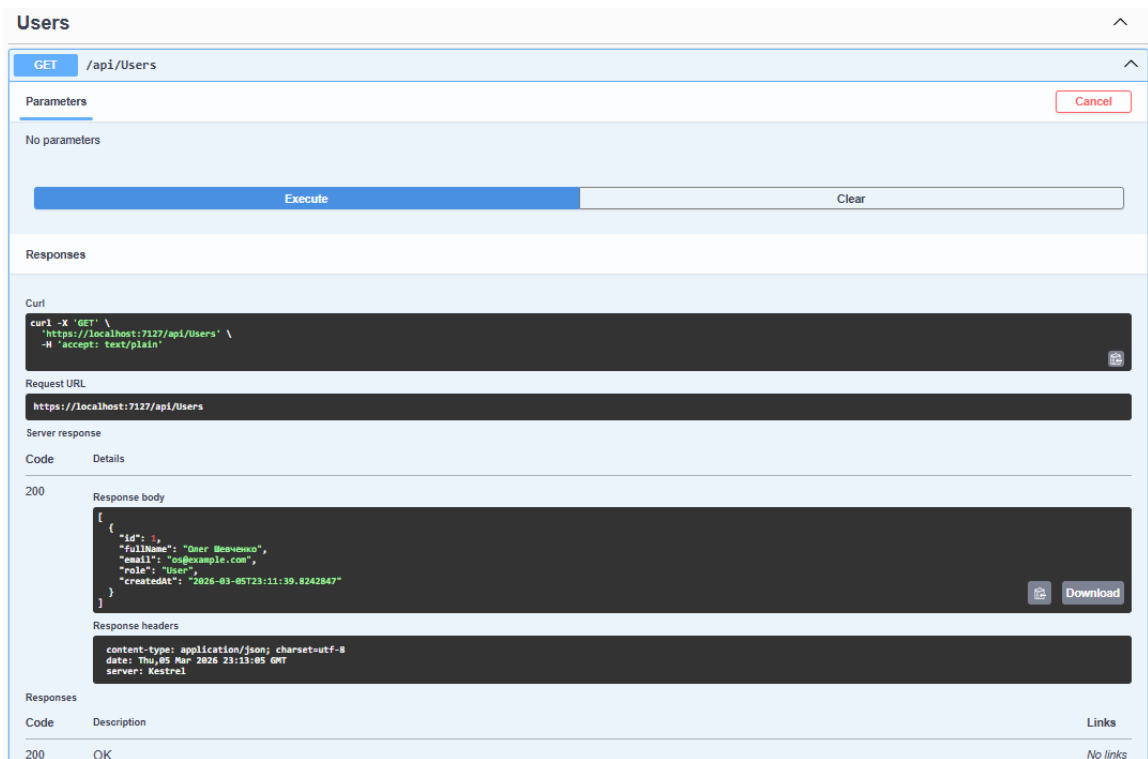


Рисунок 18 – Приклад застосування команди GET для Users.

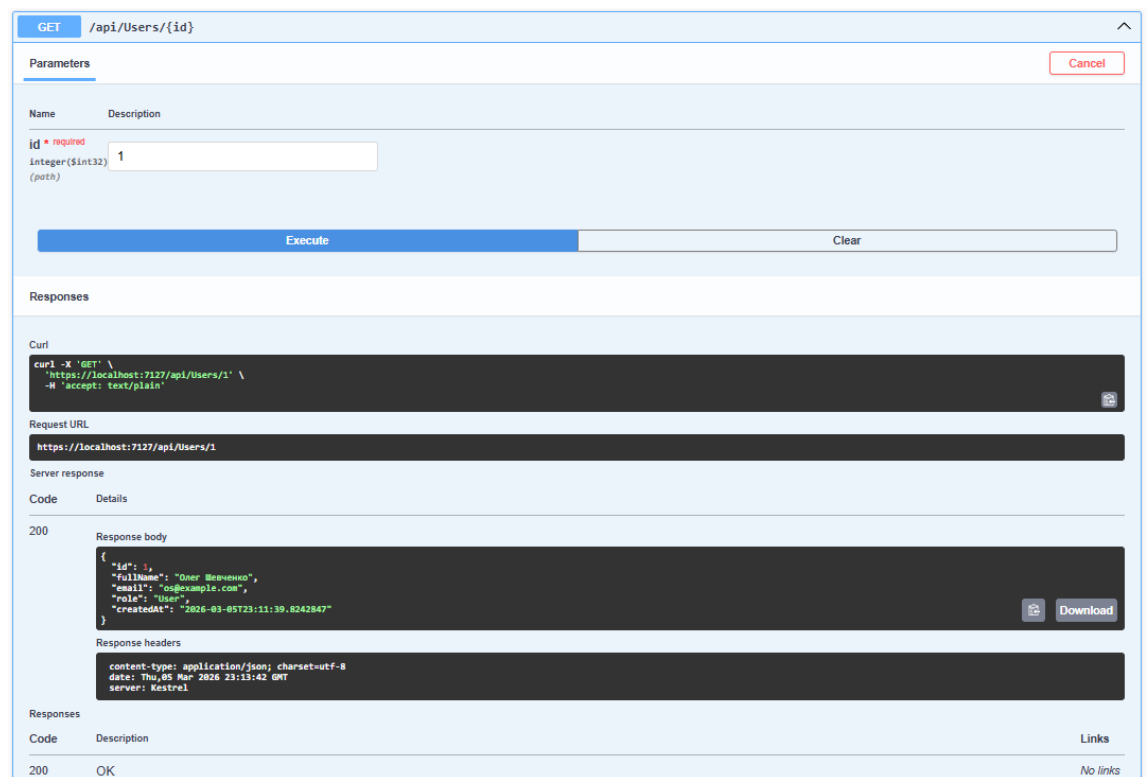


Рисунок 19 – Приклад застосування команди GET {id} для Users.

PUT /api/Users/{id}

Parameters

Name	Description
id * required	
integer(\$int32)	1
(path)	

Request body

application/json

```
{  "fullName": "Олег Шевченко",  "email": "54os@example.com"}
```

Execute Clear

Responses

Curl

```
curl -X 'PUT' \  https://localhost:7127/api/Users/1 \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "fullName": "Олег Шевченко",  "email": "54os@example.com"  }'
```

Request URL

https://localhost:7127/api/Users/1

Server response

Code Details

204
Undocumented Response headers

Рисунок 20 – Приклад застосування команди PUT {id} для Users.

POST /api/Users

Parameters

No parameters

Request body

application/json

```
{  "fullName": "Микола Горонь",  "email": "mh54@example.com",  "password": "12345",  "roleId": 1}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \  https://localhost:7127/api/Users \  -H 'accept: text/plain' \  -H 'Content-Type: application/json' \  -d '{  "fullName": "Микола Горонь",  "email": "mh54@example.com",  "password": "12345",  "roleId": 1  }'
```

Request URL

https://localhost:7127/api/Users

Server response

Code Details

201
Undocumented Response body

```
{  "id": 2,  "fullName": "Микола Горонь",  "email": "mh54@example.com",  "role": "User",  "createdAt": "2026-03-05T23:25:57.4066386Z"}
```

Download

Рисунок 21 – Створення тимчасового користувача для видалення.

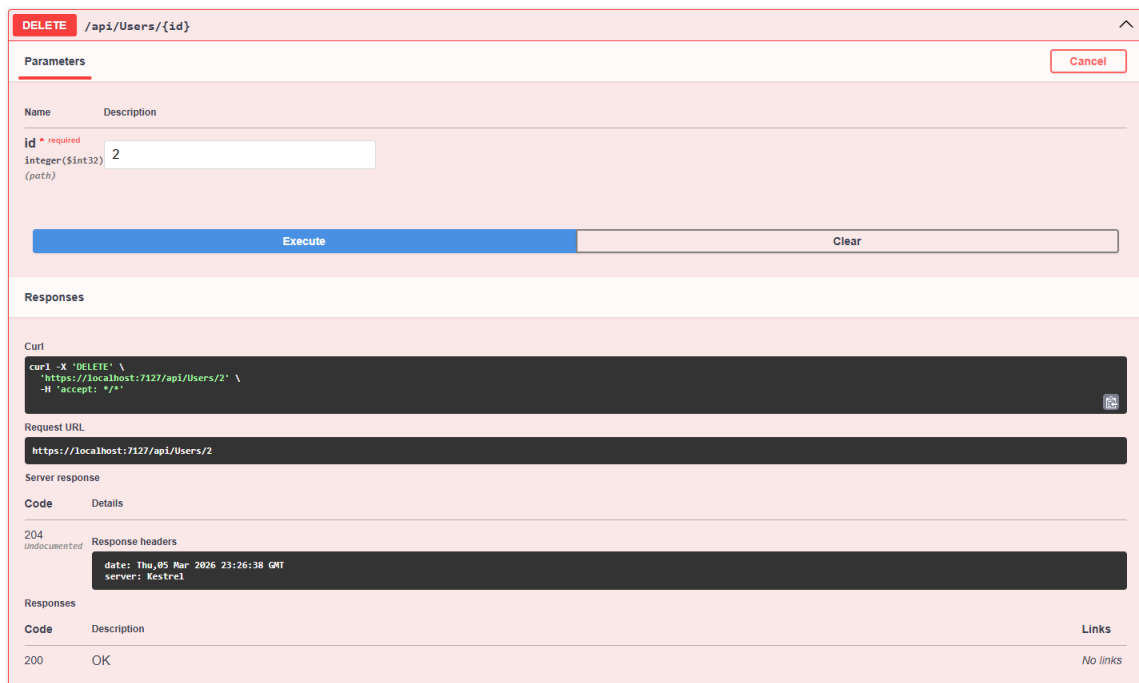


Рисунок 22 – Приклад застосування команди DELETE {id} для Users.

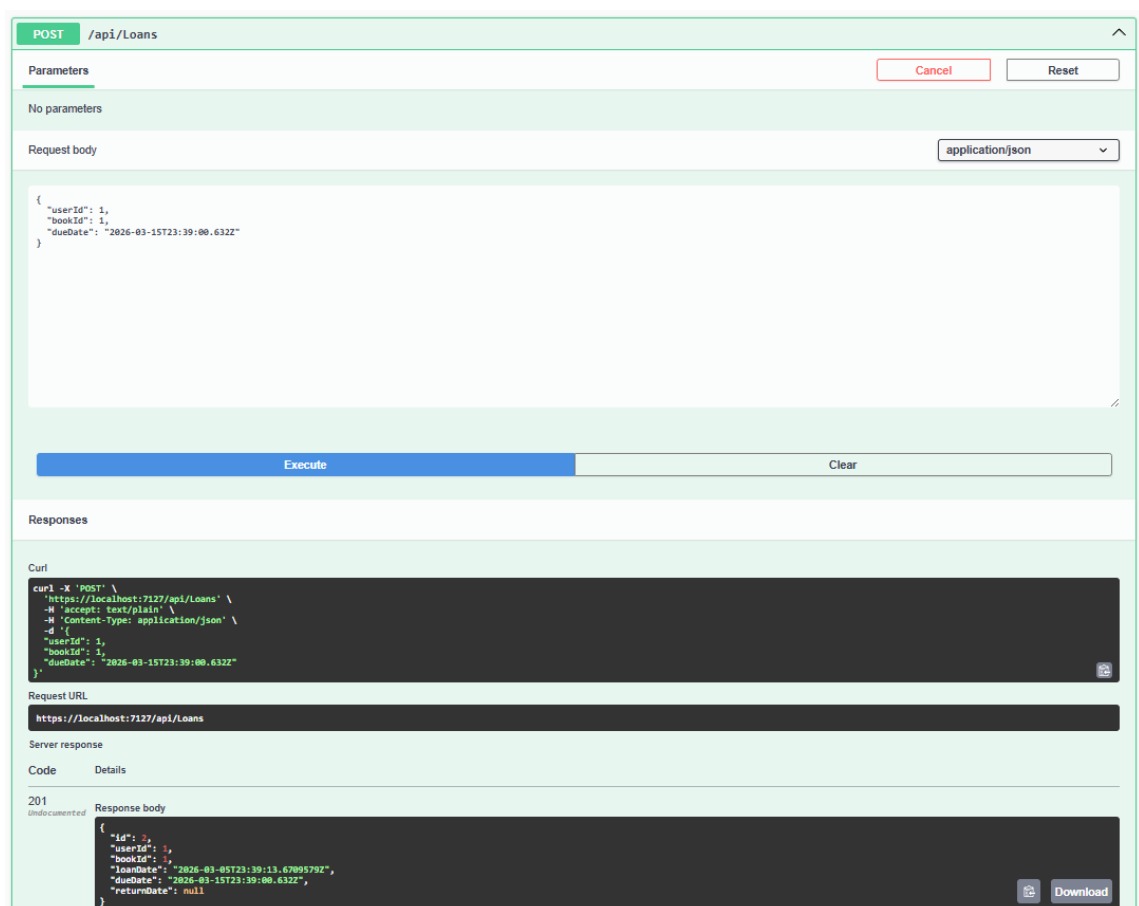


Рисунок 23 – Приклад застосування команди POST для Loans.

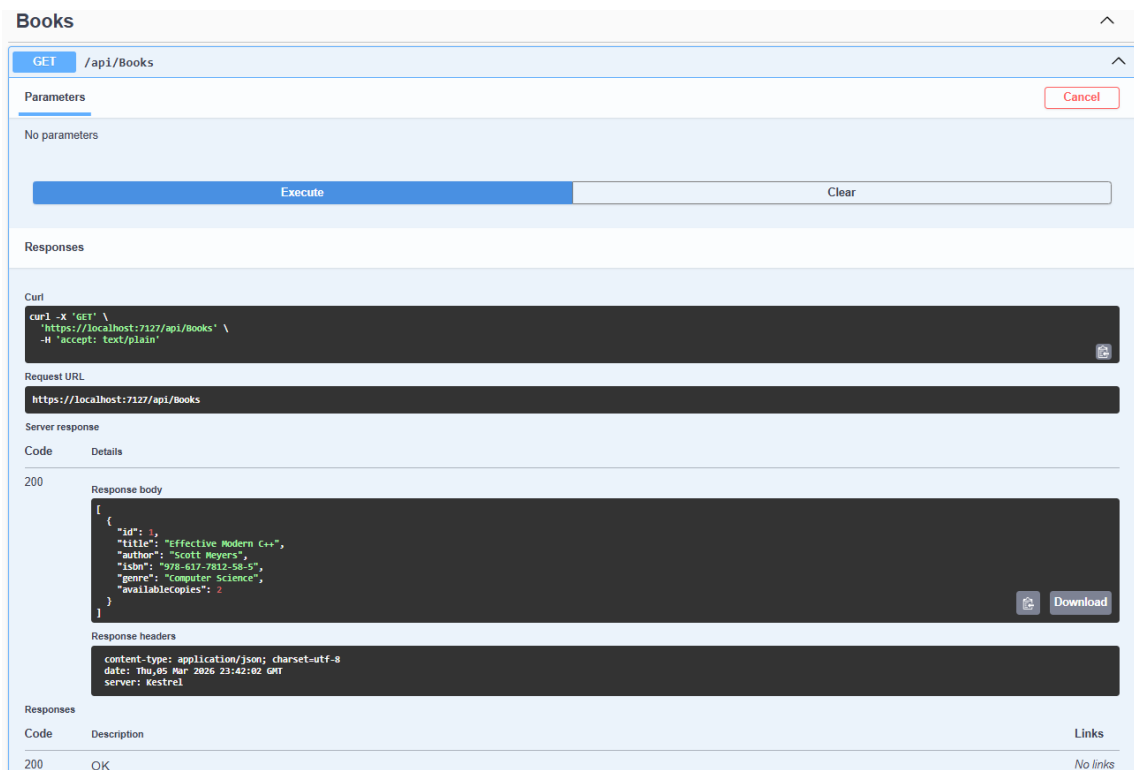


Рисунок 24 – Зміна кількості книг, після створення елемента в Loans.

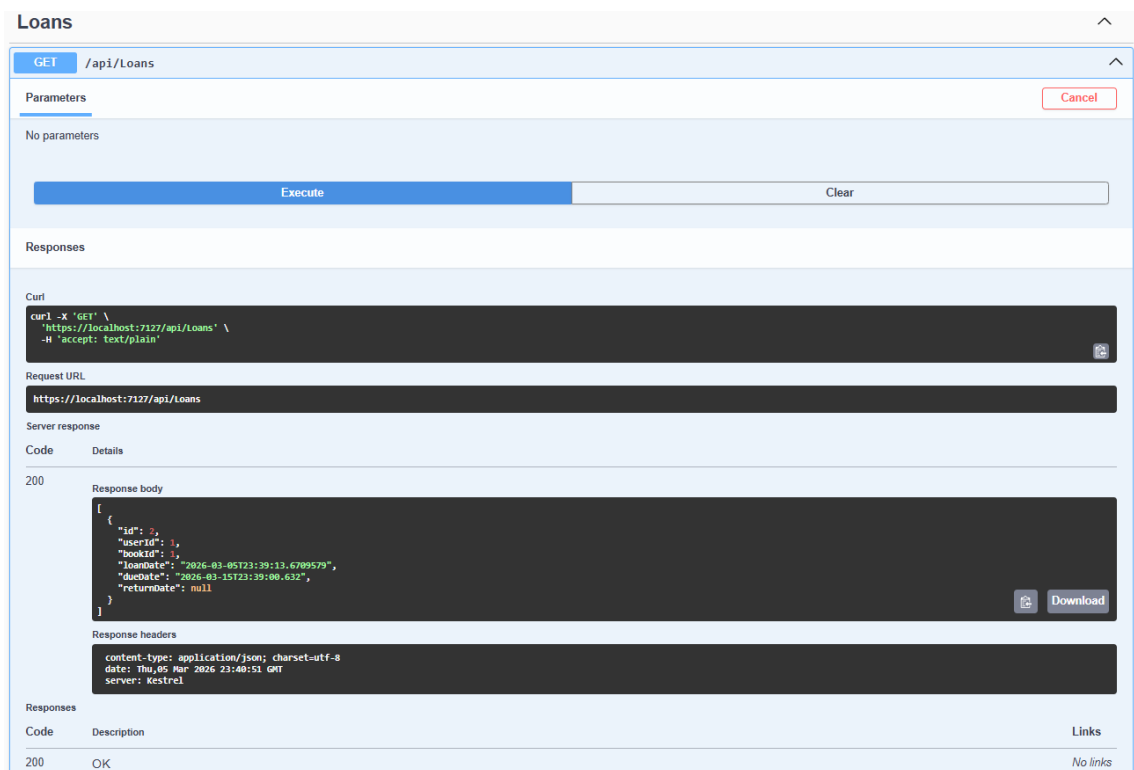


Рисунок 25 – Приклад застосування команди GET для Loans.

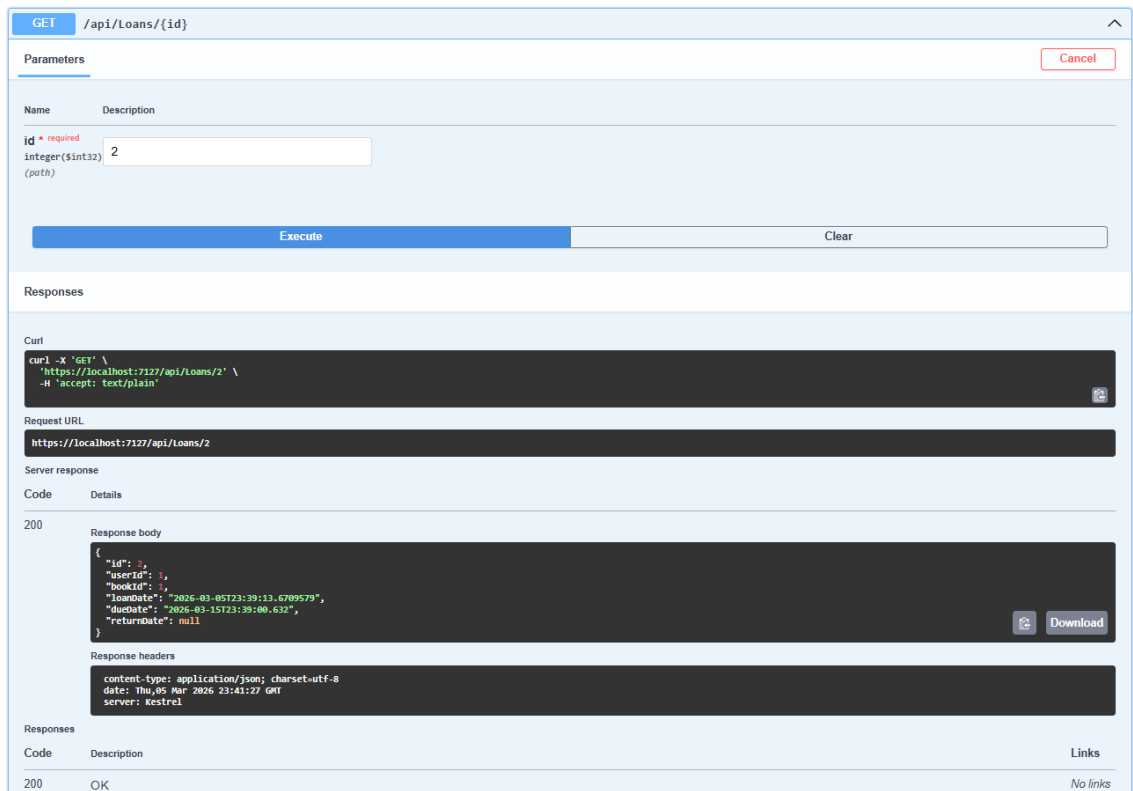


Рисунок 26 – Приклад застосування команди GET {id} для Loans.

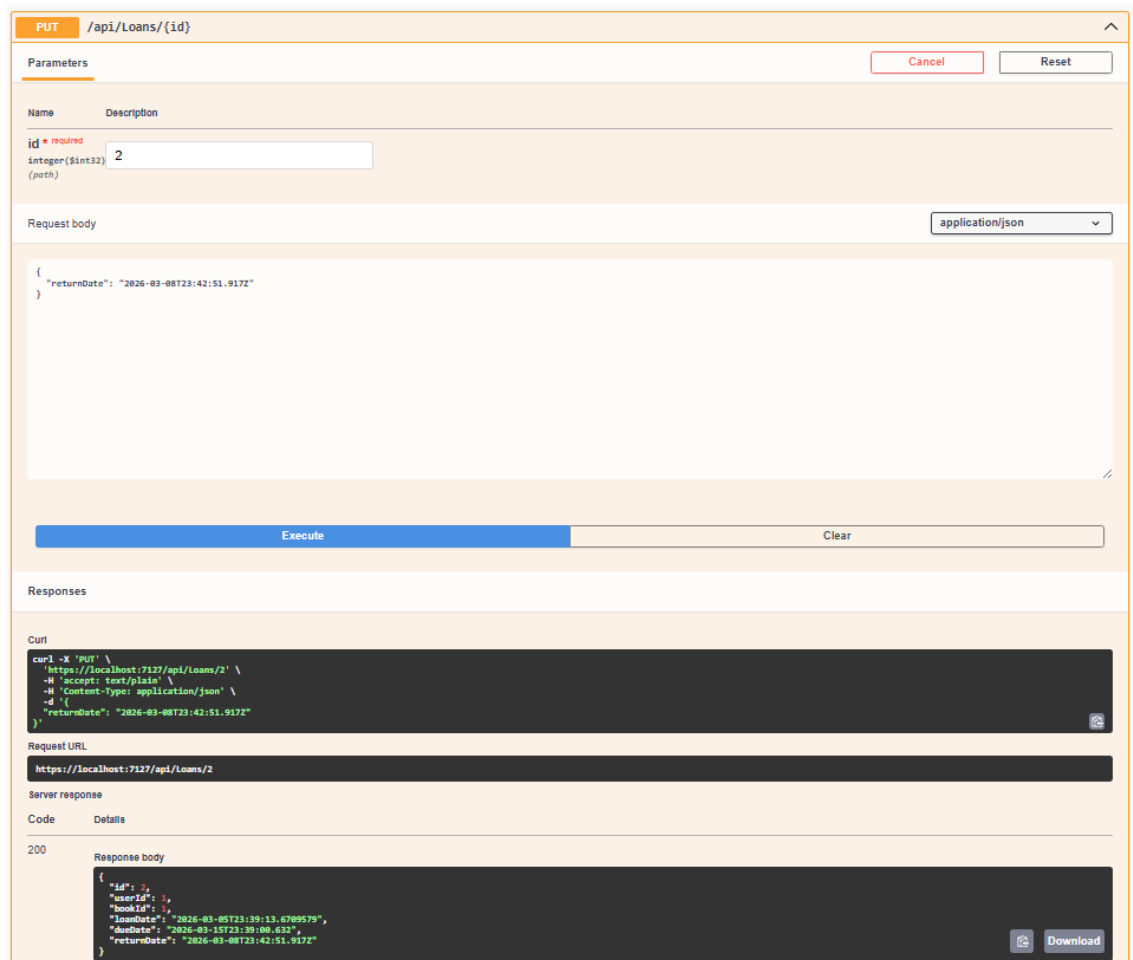


Рисунок 27 – Приклад застосування команди PUT {id} для Loans.

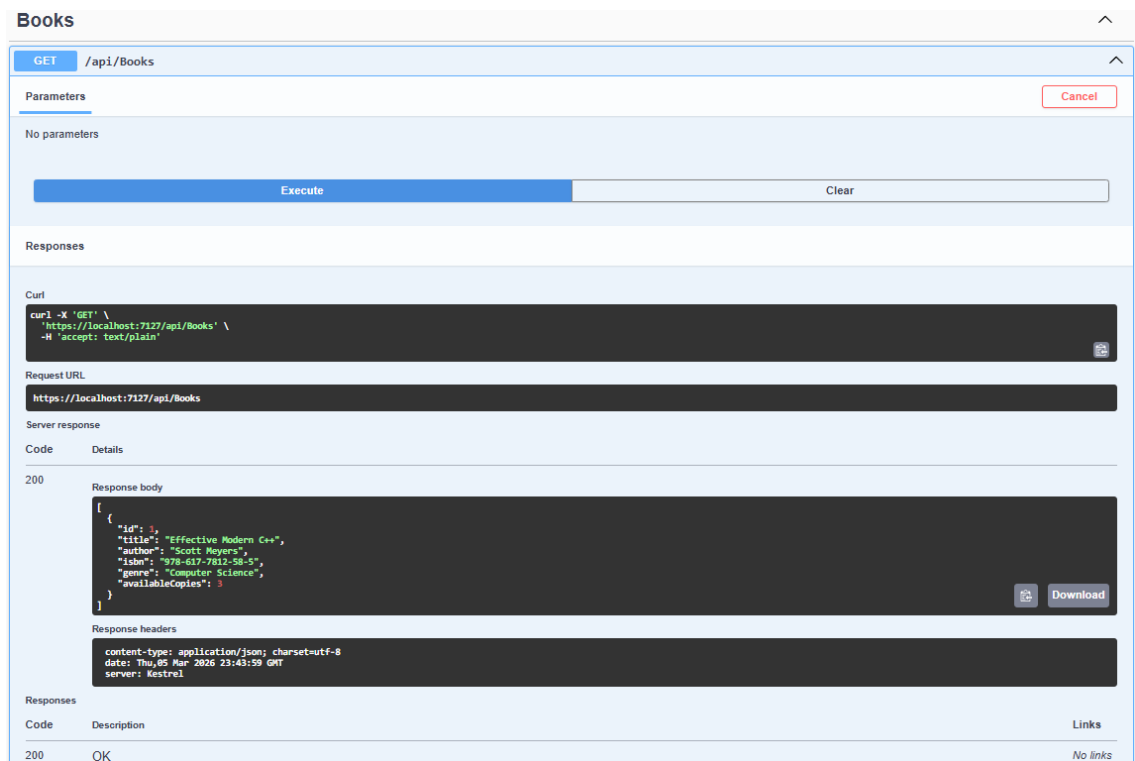


Рисунок 28 – Зміна кількості книг, після повернення книги.

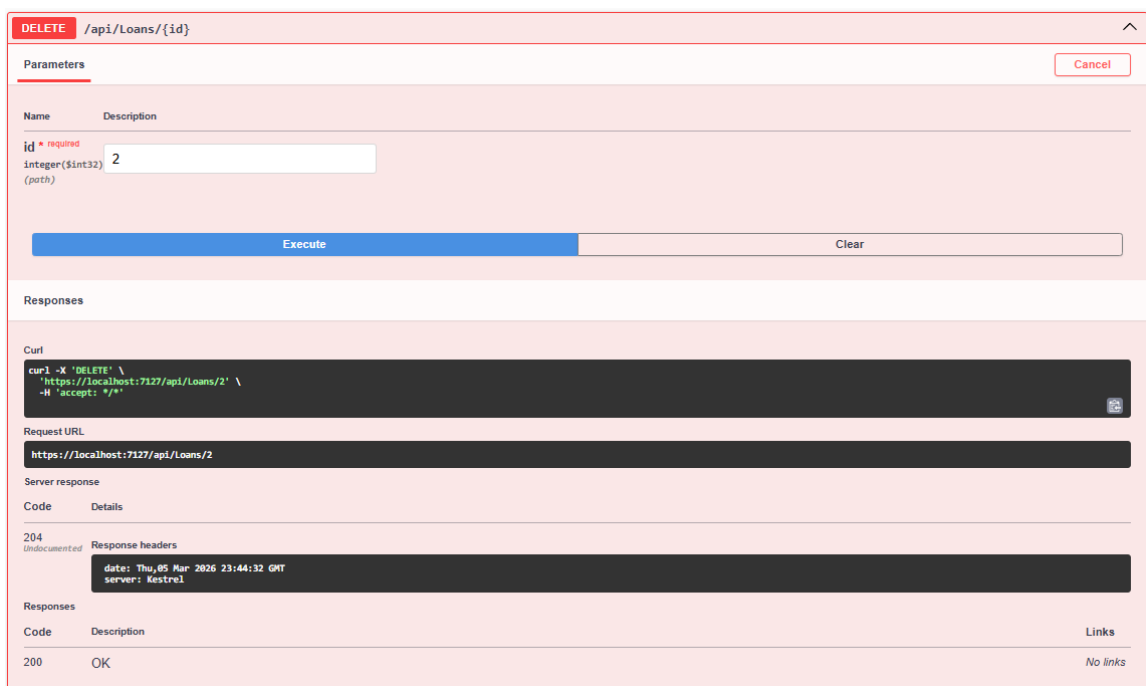


Рисунок 29 – Приклад застосування команди DELETE {id} для Loans.